

Donut Crash 최종 보고

팀 비빔밥

목차

① 발표 개요

팀 구성
발표 목적

③ 결과물

게임 시연영상
주요 파트 기획 의도

② 프로젝트 목표

MVP 스코프 설정
협업 로드맵
핵심 구현기능

④ 프로젝트 회고

발생 문제와 대응

> 발표 개요

● 프로젝트 명: Donut Crash

● 기간: 2025/10/16 – 2025/12/10

● 발표자: 강범준

● 팀 구성

개발팀

이름	포지션
임승빈	프로그래밍 팀장
김재언	프로그래밍 부팀장
최필규	UI/UX 메타 프로그래머
박인우	아웃게임 메타 프로그래머
황보승재	인게임 배틀 프로그래머
유승우	네트워크 프로그래머

기획팀

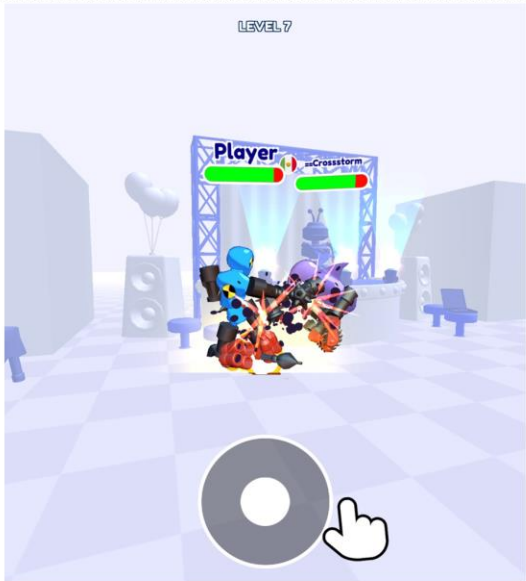
이름	포지션
황태주	기획 PM
강범준	PD, 아웃게임 시스템 기획자
서윤준	인게임 배틀 시스템 기획자 , 레벨 디자인 기획자
배성우	UI/UX 기획자, 콘텐츠 기획자

● 발표 목적

- 프로젝트 결과물 소개
- 아웃게임·인게임 기획 의도 설명
- 프로젝트 진행 과정·문제 해결과정 공유

> 프로젝트 목표

● 가이드 미션 요약

프로그램 소개	도넛 배틀은 지우개 따먹기, 레슬링과 같은 방식 으로 승부를 겨루어 상대방의 도넛을 빼앗아 오는 게임입니다.
레퍼런스	
필수 유저 체험 및 기획요소	위치, 타이밍을 맞추어 승리를 획득하는 턴제 배틀 게임 획득한 도넛을 머지 게임으로 업그레이드 하는 하이브리드
필요로 하는 기능	머지 게임의 기획과 구성 / 에셋 템플릿 필요시 구매, 익숙한 기획이 가지는 장점 을 취하고 싶습니다.

● 일부 항목 재정의 후 컨펌

기존 가이드가 지향하던 게임의 뉘앙스는 유지하되,
개발 직관성 · 콘텐츠 확장성 · 게임성
 을 강화한 신규 아이디어를 제시

변경사항 1

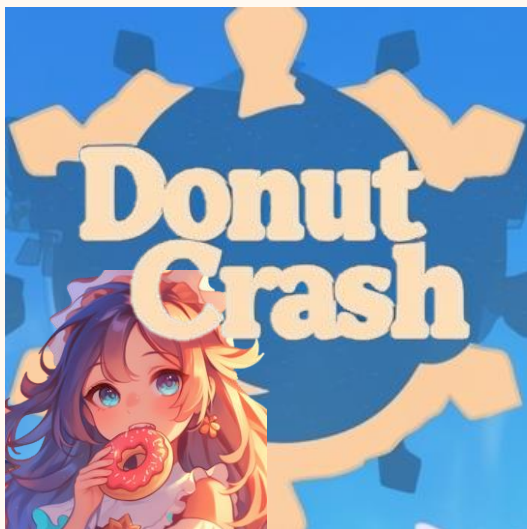
기존	지우개 따먹기, 레슬링 방식
변경	알까기 방식
사유	같은 물리액션이지만 판정 구조가 단순함

변경사항 2

기존	머지게임 하이브리드
변경	머지게임을 아웃게임으로 분류 후 오브젝트 강화 형태로 재해석
사유	반복 플레이 및 성장 구조 정립

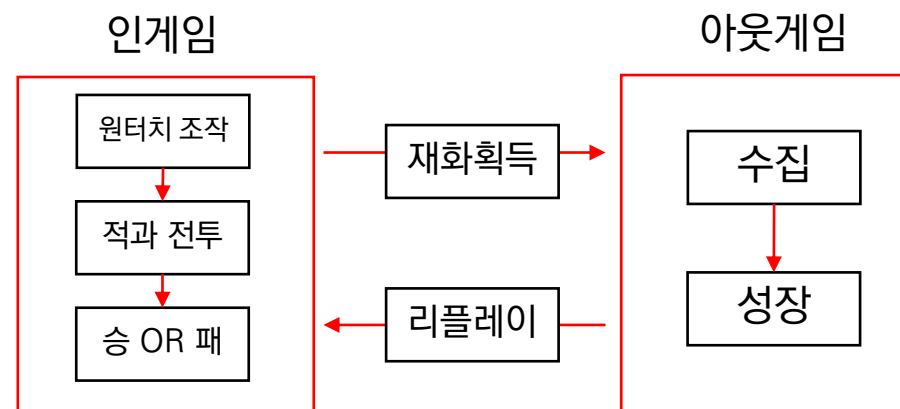
> 프로젝트 목표

● 게임 소개



게임명	Donut crash
장르	슈팅 액션 모바일게임
플랫폼	모바일 (Android / IOS)
캐치프레이즈	“튀기고, 부딪히고, 합쳐서 강해져라!” “맛있고 재밌는 PvP 알까기 배틀”

● 초기 목표 설정 (MVP 스코프)



> 프로젝트 목표

● 협업 로드맵

10월

				16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

■ 기획 - 통합 기획서(시스템,와이어프레임,DB) 작성

■ 개발 - 시스템 프레임워크 설계

■ 개발 - MVP 스코프 개발

■ 기획 - 기획서 디벨롭

■ 기획, 개발 - QA

■ 기획 - 레벨 디자인, UI 디자인

■ 개발 - 부가기능 개발

■ 빌드 제출

11월

						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30						

> 프로젝트 목표

● 협업 로드맵

12월

	1	2	3	4	5	6
7	8	9	10	11	12	13

■ 기획 - 통합 기획서(시스템,와이어프레임,DB) 작성

■ 개발 - 시스템 프레임워크 설계

■ 개발 - MVP 스코프 개발

■ 기획 - 기획서 디벨롭

■ 기획, 개발 - QA

■ 기획 - 레벨 디자인, UI 디자인

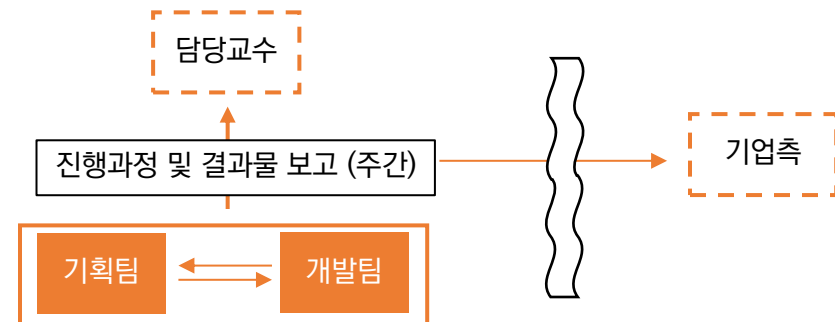
■ 개발 - 부가기능 개발

■ 빌드 제출

● 기획 - 개발 협업 파이프라인

DB	Google Spreadsheet Firebase 연동 업로더
형상관리	Sourcetree
협업	• 문서: Notion • 리포트: Notion, Discord • 자료공유: Google Drive • 디자인: Unity 스토어 에셋, Figma
일정관리	Notion
비고	매일 오전 스크럼

● 피드백 및 폴리싱



> 프로젝트 목표

● 핵심 구현기능 정리

아웃게임 (9)

계정, 네트워크, 데이터 관리

- Firebase 로그인
- 리더보드 시스템
- 유저 데이터 관리
- 구글 플레이 게임즈 연동

메타 루프

- 텍 시스템
- 상점 시스템 (IAP 포함)
- 도넛 머지 시스템
- 도넛 제작 시스템
- 도넛 해금 트리 시스템

인게임 (14)

코어 루프

- Photon 네트워크 시스템
- 매치메이킹 시스템
- 드래그 슬링 조작
- 스플라인
- Rigidbody2D 기반 충돌 물리
- Unity 2D 물리
- 턴제 + 턴제한시간
- 도넛교체, 텍 대기열
- 스탯 기반 데미지 (공/체/방/질량/ 치명)
- 턴 알림
- 도넛 스킬
- 충돌 이펙트
- BGM, SFX
- 마녀 제빵사 스킬

공통 (1)

환경설정 시스템

> 결과물 - 시연 영상



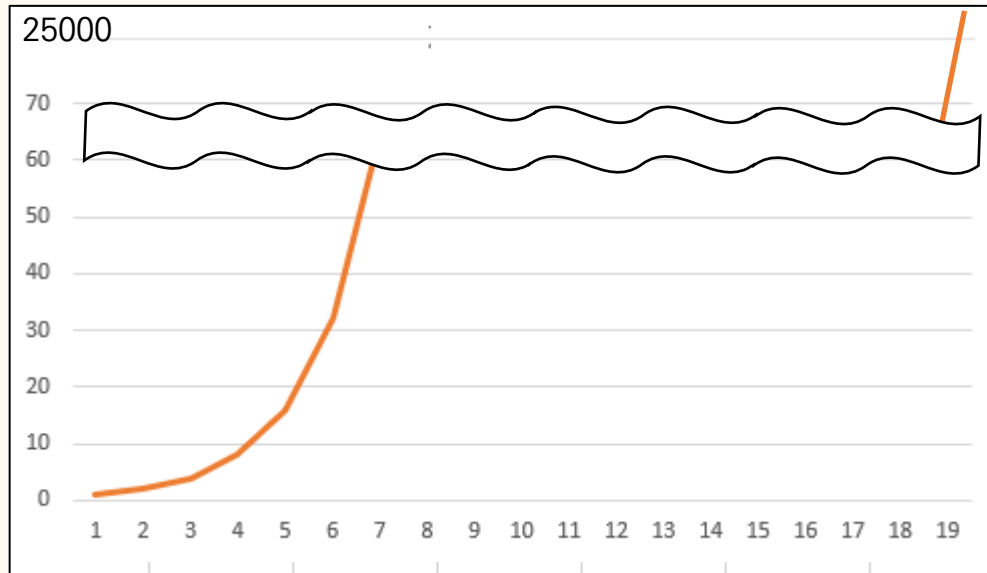
> 결과물 - 주요 파트 기획 의도



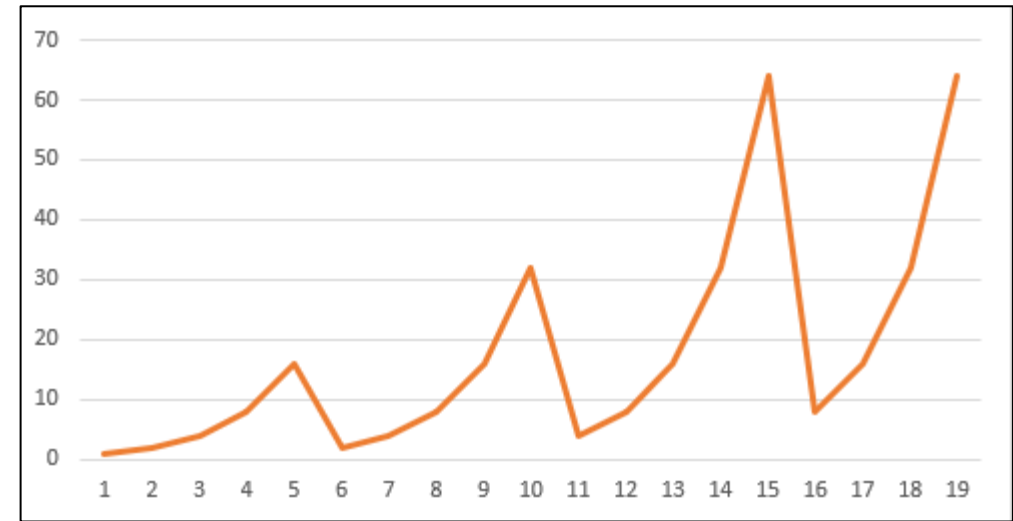
섹션 분류	아웃게임
이름	도넛 머지 합성 시스템
기획 리빙 포인트	일반 머지 시스템의 2ⁿ 구조로 인한 재료 폭증 문제를 구조적으로 해소 5레벨 이후부터 재료 도넛 개수를 의도적으로 '하향 반복'시키는 설계 결론적으로 총 20레벨까지 상승하는 머지 구조 완성
기대효과	유저 성장 체감은 유지하면서 강화 피로도는 급격히 낮추는 곡선 제어 구조 고레벨 구간에서도 강화 진입 장벽을 낮추어 장기 플레이를 유지시킴

> 결과물 - 주요 파트 기획 의도

● 재료 도넛 개수의 하향 반복 구조란?



일반 머지 그래프



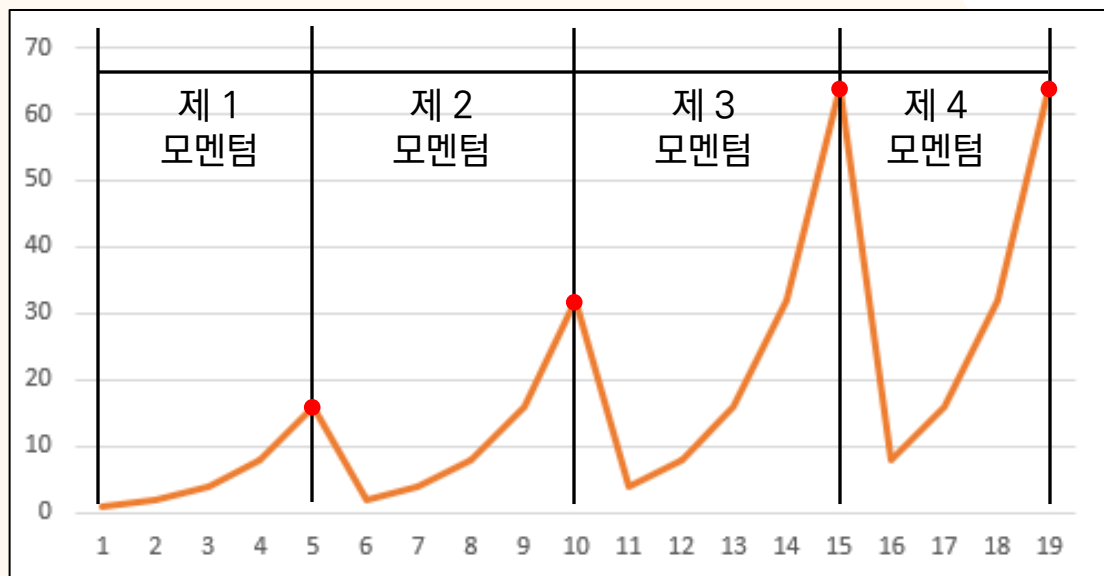
조정된 머지 그래프

*가로 축은 강화레벨, 세로 축은 필요 재료 도넛의 개수(1레벨)

재료 도넛 소모량이 기하급수적으로 폭증하는 기존 머지 구조의 한계를
조정된 가변 재료 레벨 설계를 통해 효과적으로 해소

> 결과물 - 주요 파트 기획 의도

● 모멘텀 별 추가 스탯 강화



조정 머지 그래프

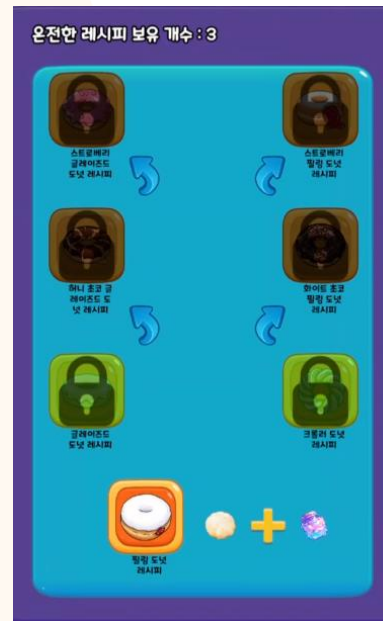
이름	타입	설명
uid	string	고유 ID
origin	string	도넛 기본데이터 ID
hpPerLevel	int	레벨당 체력 증가량
hpBonus	int	5레벨당 체력 추가 증가량
defPerLevel	int	레벨당 방어력 증가량
defBonus	int	5레벨당 방어력 추가 증가량
atkPerLevel	int	레벨당 공격력 증가량
atkBonus	int	5레벨당 공격력 추가 증가량
critPerLevel	float	레벨당 크리티컬 확률 증가량
critBonus	float	5레벨당 크리티컬 확률 추가 증가량
massPerLevel	float	레벨당 질량 증가량
massBonus	float	5레벨당 질량 추가 증가량

도넛 레벨당 스탯 증가량 DB

재료 도넛 소모가 정점을 찍고 다시 완화되는 4개의 구간을 모멘텀 이라고 명명

각 모멘텀 구간 종료 시점마다 스탯 추가 성장 보상 제공

> 결과물 - 주요 파트 기획 의도



섹션 분류	아웃게임
이름	도넛 제작·해금 시스템
기획 리빙 포인트	하이퍼 캐주얼 원터치 게임 특성상 아웃게임 피로도를 최소화하기 위해 제작·해금 구조를 최대한 단순화
기대효과	성장 루프 구성은 다양하게 부담은 가볍게 설계하여스트레스 없이 입체적인 플레이 경험을 제공

> 결과물 - 주요 파트 기획 의도

● 제작식 설계의 일반적인 접근



ChocoDough, Cacao, SugarPowder, Butter



PlainDough, StrawberryJam, SweetCrem



PlainDough, BerryExtract, sugarGlaze



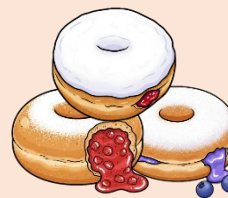
@@@@, #####, %%%%, &&&&

모든 도넛의 제작식을 각각 설계, 재료 종류 방대

● 새로 고안한 제작식 설계



도넛 반죽 + 마법 재료 라는 컨셉
제작식과 제작식을 구성하는 재료 단순화



도넛을 이미지에 맞게 3개의 계열로 분류 후
같은 계열의 도넛은 제작식 통일

> 결과물 - 주요 파트 기획 의도

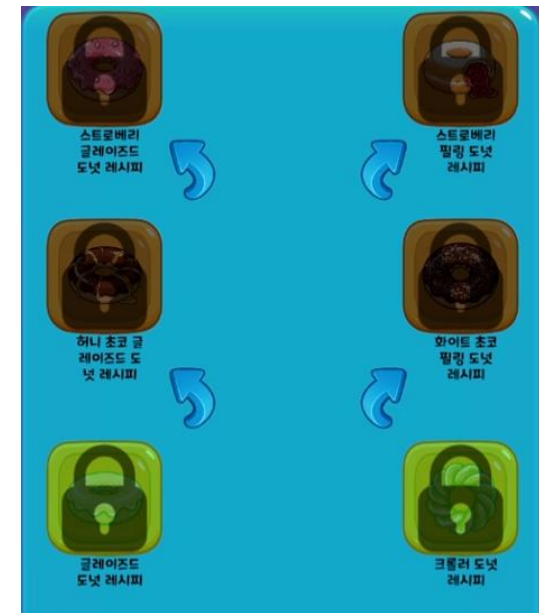
●도넛 해금 구조



레시피 조각 수집



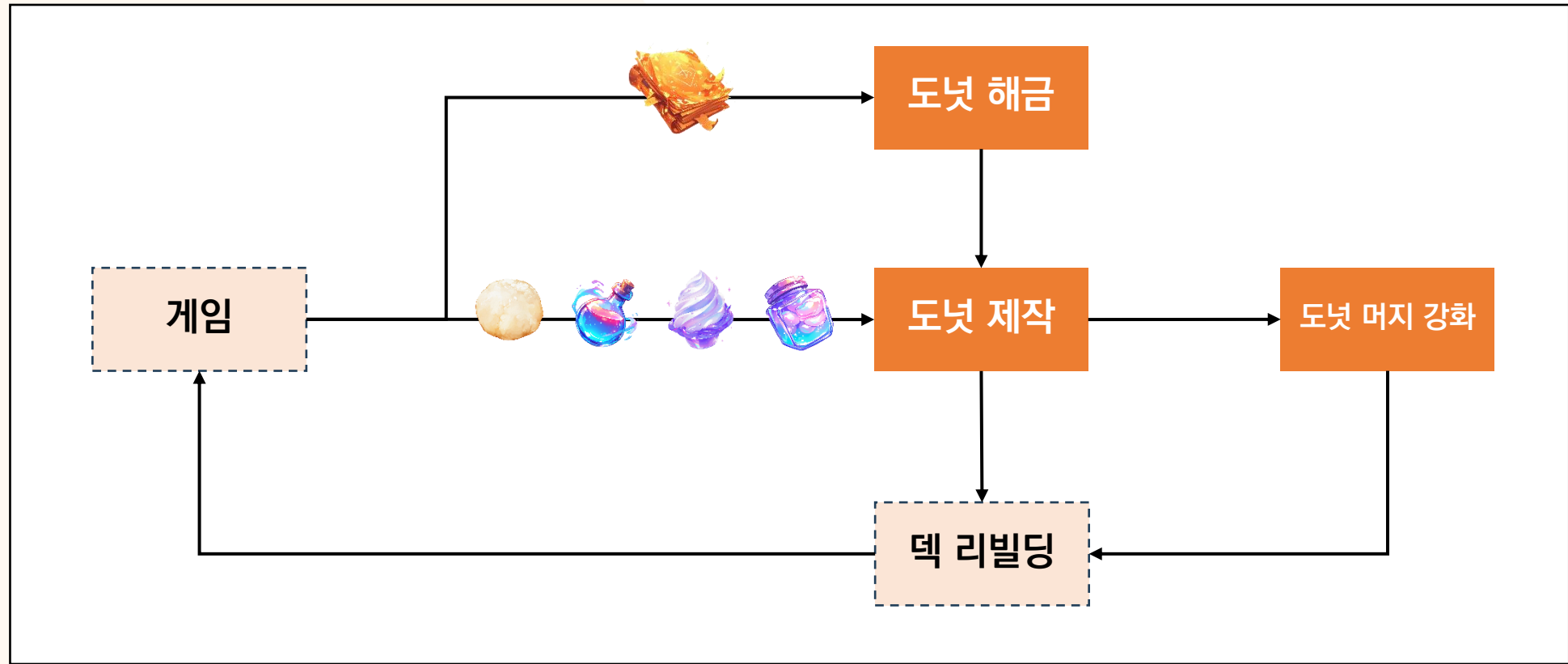
만능 레시피 교환



티어별 순차 해금

단 하나의 레시피로 모든 도넛 해금 가능 구조

> 결과물 - 주요 파트 기획 의도



성장 루프 구성

성장 루프 구성은 다양하게, 부담은 가볍게 설계하여 스트레스 없이 입체적인 플레이 경험을 제공

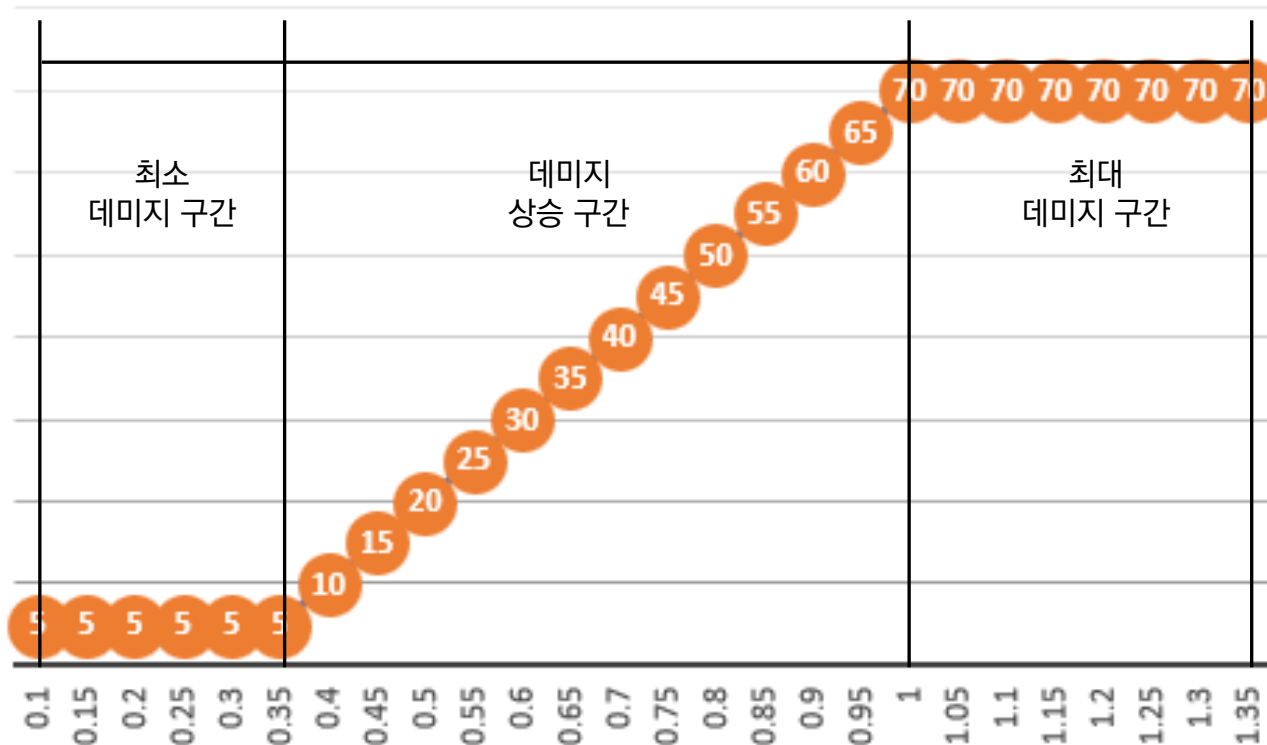
> 결과물 - 주요 파트 기획 의도



섹션 분류	인게임
이름	데미지 시스템
기획 리빙 포인트	물리 속도를 0~1로 정규화하여 조작 → 충돌 속도 → 데미지 로 직관적 연결 최소 데미지 보장 + 크리티컬 후처리로 방어·속도·크리 간 상호작용 구조 완성
기대효과	데미지 체감과 실제 수치의 괴리 해소 조작 강도에 따른 피해 피드백이 예측 가능해짐

> 결과물 - 주요 파트 기획 의도

●도넛 스피드와 데미지의 상관관계



도넛 스피드, 데미지 상관관계 그래프

*상황: 공격력(ATK)이 70인 도넛이 방어력(DEF)이 30인 도넛을 가격

*가로 축은 슬링조작 시 도넛 스피드(0~1), 세로 축은 적용되는 데미지

●적용 데미지 공식

$$SPD = \text{clamp01}((x - \min) / (\max - \min))$$

$$DMG = \max(ATK * SPD - DEF, \text{최소데미지})$$

최소 데미지 구간

충돌 속도가 낮아 $ATK \times SPD < DEF$ 인 구간
실제 데미지는 최소 보정값(5)으로 고정
약하게 부딪혀도 항상 일정 피해는 발생하여
전투가 정체되지 않도록 보장

데미지 상승 구간

$ATK \times SPD$ 가 DEF를 초과하는 시점부터 시작
속도(SPD)에 비례해 데미지가 선형으로 증가
조작 강도와 피해량이 직관적으로 연결되는
핵심 체감 구간

최대 데미지 구간

SPD가 1에 도달한 구간
최대 공격력 기반의 상한 데미지에 도달
강한 조작 보상이 명확히 체감되는 피니시 영역

> 결과물 - 주요 파트 기획 의도



섹션 분류	인게임
이름	도넛 티어별 성능 설정
기획 리빙 포인트	1티어는 '학습용', 2티어는 '전략 확장', 3티어는 '완성형 스킬' 구조로 단계적 진화 버프·디버프·스택형 스킬 간 상호작용으로 '타이밍 싸움' 중심의 전투 구조를 구축
기대효과	스킬 조합에 따른 전투 변수 증가 → 리플레이성 대폭 강화 상위 티어 도넛은 하위 티어의 모든 성능을 포괄하는 완전한 상위 단계로 설계하여, 유저에게 명확한 성장 목표를 제공

> 결과물 - 주요 파트 기획 의도

●도넛 스킬 발동 타이밍 분류

목차	스킬 활성화 타이밍
1	스폰 시
2	해당 아군 도넛의 턴 시작 시
3	해당 아군 도넛의 턴 진행중에
4	해당 아군 도넛의 턴이 끝날 때
5	내 팀 도넛의 턴 시작 시
6	내 팀 도넛의 턴 진행중에
7	내 팀 도넛의 턴이 끝날 때
8	해당 적군 도넛의 턴 시작 시
9	해당 적군 도넛의 턴 진행중에
10	해당 적군 도넛의 턴이 끝날 때

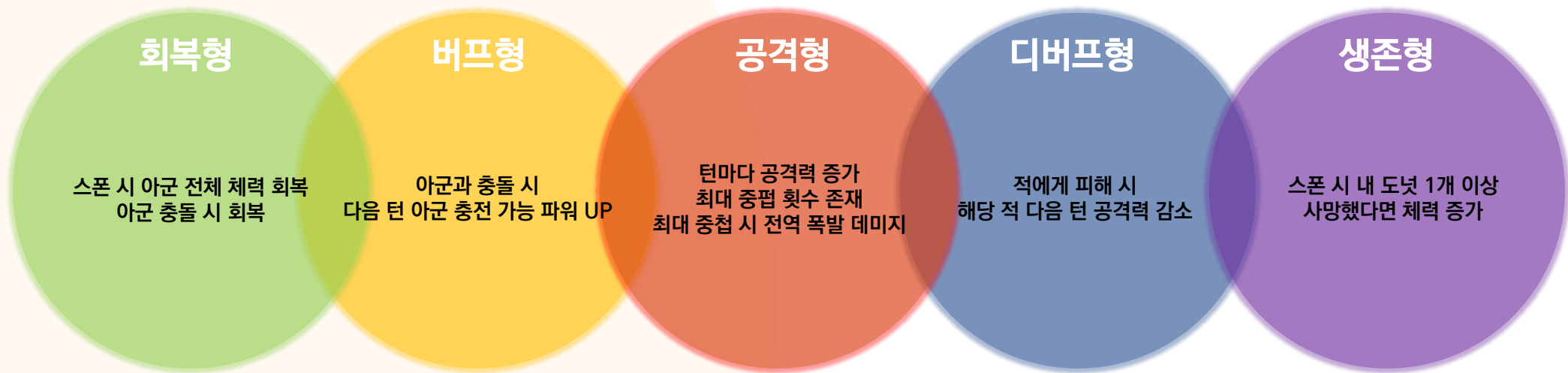
●도넛 스킬 효과 파라미터 분류

목차	스킬 효과 파라미터
1	공격력
2	체력 회복
3	체력 증가
4	방어력
5	슬링 파워
6	도넛 반동 스피드

전투 변수를 확대하기 위해 스킬 발동 조건과 효과 파라미터를 세분화하여 설계

> 결과물 - 주요 파트 기획 의도

●도넛 스킬 타입



*인게임에 실제 적용되는 타입별 스킬 예시

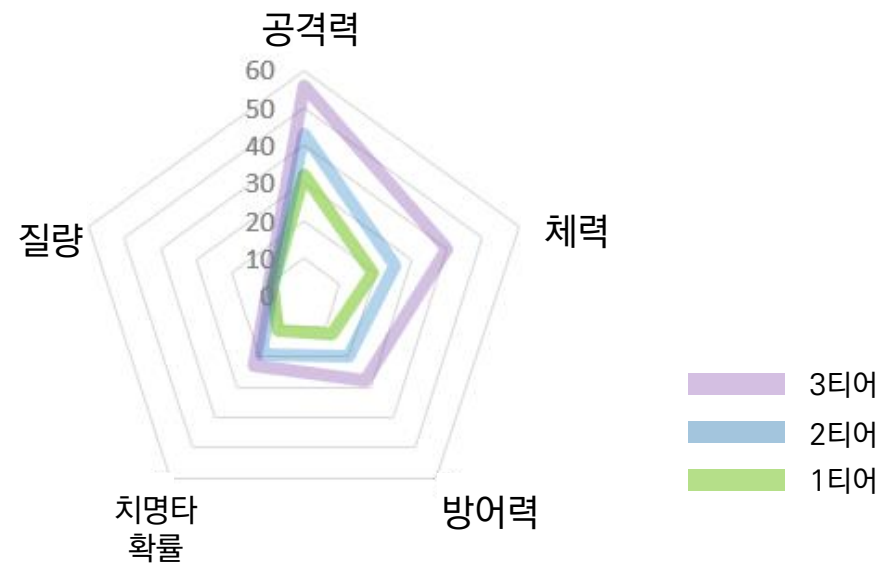
전투흐름과 역할에 맞춰 다양한 성격의 스킬 타입을 체계적으로 구성

> 결과물 - 주요 파트 기획 의도

● 도넛 티어별 해금 구조



● 도넛 티어별 1레벨 평균 파라미터 비교



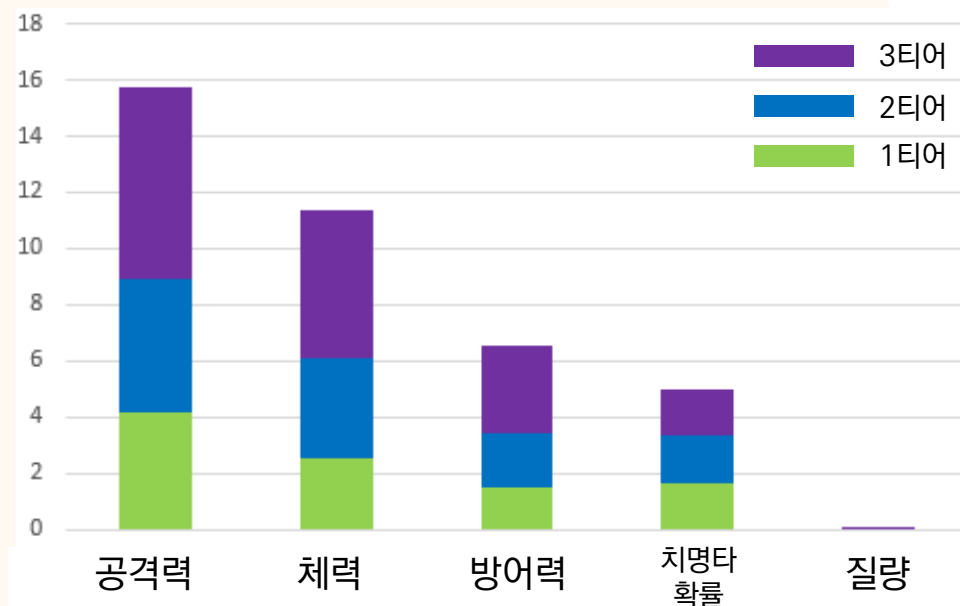
도넛 티어별 1레벨 평균 스펙 파라미터

*비교 가독성을 위해 파라미터별 수치 스케일을 보정하여 표기함
(공격력·방어력·질량은 원값 유지, 체력은 1/3 스케일 적용, 치명타 확률은 % 기준 사용)

해금 트리 구조로 높은 티어의 도넛을 순차적으로 획득, 점진적 성장 구조 확보

> 결과물 - 주요 파트 기획 의도

●도넛 티어별 레벨당 평균 성장 척도 비교



도넛 티어별 레벨당 평균 성장 척도 그래프

*비교 가독성을 위해 파라미터별 수치 스케일을 보정하여 표기함
(공격력·방어력·질량은 원값 유지, 체력은 1/3 스케일 적용, 치명타 확률은 % 기준 사용)

●도넛 티어별 스킬 파워 비교

1 TEAR

헬즈도어 LV.1

턴이 지날 때 마다 공격력 1 증가, 최대 5중첩

2 TEAR

헬즈도어 LV.2

턴이 지날 때 마다 공격력 2 증가, 최대 5중첩

3 TEAR

헬즈도어 LV.3

턴이 지날 때 마다 공격력 3 증가, 최대 5중첩

*일부 예시, 모든 스킬이 레벨로 파워가 분화되는 것은 아님

높은 티어의 도넛은 낮은 티어 도넛의 완전한 상위호환 → 명확한 성장 목표 제공

> 프로젝트 회고

● 프로젝트 진행 중 주요 발생 문제와 대응

문제 이름	Firestore 이슈 Realtime Database 서비스 자체의 최적화로 인한 실시간 동기화 방식에 이슈가 존재
문제 원인	Realtime Database의 경우 서버와 통신하는 것이 아닌 로컬에 일정 시간마다 데이터를 가져온 뒤 해당 데이터를 활용하는 방식으로 비용 절감과 최적화를 하는 것으로 확인됨 그에 따라 특정 조건에 데이터가 최신 데이터 라는것을 보장하지 않는 상황
문제 해결 대상	개발팀
해결 방법	테이블 및 유저 데이터의 경우 Firestore로 데이터베이스 이전과 서버와의 직접 통신 로직을 구성해 해결

최신 데이터 유지

Firebase 실시간 데이터베이스는 활성 리스너의 데이터를 동기화하고 로컬 사본을 저장합니다. 또한 특정 위치의 동기화를 유지할 수 있습니다.

```
final scoresRef = FirebaseDatabase.getInstance().ref("scores");  
scoresRef.keepSynced(true);
```

Firebase 실시간 데이터베이스 클라이언트는 참조에 활성 리스너가 없어도 이러한 위치의 데이터를 자동으로 다운로드하고 동기화합니다. 동기화를 해제하려면 다음 코드를 사용합니다.

```
scoresRef.keepSynced(false);
```

기본적으로 이전에 동기화한 데이터 중 10MB가 캐시됩니다. 대부분의 애플리케이션에서는 이 용량으로 충분합니다. 구성된 크기보다 캐시가 커지면 Firebase 실시간 데이터베이스가 가장 오래전에 사용된 데이터를 삭제합니다. 동기화가 유지되는 데이터는 캐시에서 삭제되지 않습니다.

Firebase 공식 문서
데이터를 항상 최신버전을 유지하지 못한다는 안내

```
hot = await _userDocs.GetSnapshotAsync(Source.Server).AsUniTask();
```

유저 데이터를 서버로부터 로드하는 함수
로컬을 사용하지 않고 서버와의 통신을 강제할 수 있음

> 프로젝트 회고

● 프로젝트 진행 중 주요 발생 문제와 대응

문제 이름	협업 중 코드 충돌
문제 원인	개발팀 간의 작업 중 서로 연관된 기능개발에 있어서 코드의 충돌로 인한 오류 발생
문제 해결 대상	개발팀
해결 방법	이벤트 형식을 통한 콜백 구조와 이벤트를 관리하는 허브를 구성해 해결

```
public class EventHub : MonoSingleton<EventHub>
```

```
{
    private readonly Dictionary<Type, Delegate> _eventDic = new();

    /// <summary>
    /// <see cref="IEvent"/> 인터페이스를 구현하는 구조체를 매개변수로 사용하는 함수들을 등록하는 함수
    /// </summary>
    /// <param name="callback">등록할 함수</param>
    /// <typeparam name="T"><see cref="IEvent"/>를 구현하는 구조체</typeparam>
    <!-- 자주 호출됨 ⓘ 263 사용 위치 ⓘ SeunbBinLim -->
    public void RegisterEvent<T>(Action<T> callback)
    {
        if (_eventDic.TryGetValue(typeof(T), out Delegate @delegate))
        {
            _eventDic.Add(typeof(T), @delegate.Combine(callback));
        }
        else
        {
            _eventDic[typeof(T)] = (Action<T>)callback;
        }
    }

    /// <summary>
    /// <see cref="IEvent"/> 인터페이스
    /// </summary>
    /// <param name="callback">등록 할 함수</param>
    /// <typeparam name="T"><see cref="IEvent"/>를 구현하는 구조체</typeparam>
    <!-- 자주 호출됨 ⓘ 264 사용 위치 ⓘ SeunbBinLim -->
    public void UnRegisterEvent<T>(Action<T> callback)
    {
        if (_eventDic.TryGetValue(typeof(T), out Delegate @delegate))
        {
            _eventDic.Remove(typeof(T));
        }
    }

    /// <summary> 상품 구매 요청 이벤트 </summary>
    ⓘ 4 사용 위치 ⓘ SeunbBinLim
    public struct RequestTryBuyEvent : IEvent
    {
        public string merchandiseId;

        <!-- 자주 호출됨 ⓘ 1 사용 위치 ⓘ SeunbBinLim -->
        public RequestTryBuyEvent(string merchandiseId) => this.merchandiseId = merchandiseId;
    }

    /// <summary> 상품 구매 성공 이벤트 </summary>
    ⓘ 5 사용 위치 ⓘ SeunbBinLim
    public struct BuySuccessEvent : IEvent
    {
        public string merchandiseId;

        <!-- 자주 호출됨 ⓘ 2 사용 위치 ⓘ SeunbBinLim -->
        public BuySuccessEvent(string merchandiseId) => this.merchandiseId = merchandiseId;
    }

    /// <summary> 상품 구매 실패 이벤트 </summary>
    ⓘ SeunbBinLim
    public struct BuyFailedEvent : IEvent
    {
        public string merchandiseId;
        public string error;

        ⓘ SeunbBinLim
        public BuyFailedEvent(string merchandiseId, string error)
        {
            this.merchandiseId = merchandiseId;
            this.error = error;
        }
    }
}
```

EventHub - 서로간의 충돌을 방지하고 이벤트 기반의 콜백을 수행하기 위한 중계기 역할을 하는 코드

EventStruct - EventHub를 통한 무분별한 통신을 방지하기 위해 IEvent 라는 인터페이스로 제약을 건 뒤 함께 전달할 데이터를 묶어놓은 컨테이너

> 프로젝트 회고

● 프로젝트 진행 중 주요 발생 문제와 대응

문제 이름	유니티 라이프 사이클 이슈 유니티 라이프 사이클을 통한 이벤트 관리 중 이벤트 등록 및 해제에 있어서 오류가 발생
문제 원인	처음부터 활성화 되어있지 않은 객체의 경우 유니티 라이프 사이클 자체가 동작하지 않는 상황
문제 해결 대상	개발팀
해결 방법	시스템과 뷰를 분리하고 활성화 상태와 상관 없이 독립적으로 이벤트 관리가 가능하도록 수정

```
private void Start()
{
    merchandiseTable = DataManager.Instance.GetMerchandiseTable().ToDictionary(m :MerchandiseData =>
    productDict = DataManager.Instance.GetProductTable().ToDictionary(p :ProductData => p.uid, p :Produ

    _newGold = DataManager.Instance.UserGold;
    _newCash = DataManager.Instance.UserCash;
    _newRecipePieces = DataManager.Instance.RecipePieces;
    _newPerfectRecipes = DataManager.Instance.PerfectRecipe;
```

```
//===== Broad Cast =====//
EventHub.Instance.RegisterEvent<DS.BroadcastSetUserGoldEvent>(OnBroadcast);
EventHub.Instance.RegisterEvent<DS.BroadcastSetUserCashEvent>(OnBroadcast);
EventHub.Instance.RegisterEvent<DS.BroadcastSetUserRecipePiecesEvent>(OnBroadcast);
EventHub.Instance.RegisterEvent<DS.BroadcastSetUserPerfectRecipeEvent>(OnBroadcast);
```

```
이벤트 함수 SeunbBinLim +1
private void OnDestroy()
{
    EventHub.Instance?.UnRegisterEvent<OGS.RequestTryBuyEvent>(OnRequestTryBuyEvent);

    //===== Broad Cast =====//
    EventHub.Instance?.UnRegisterEvent<DS.BroadcastSetUserGoldEvent>(OnBroadcast);
    EventHub.Instance?.UnRegisterEvent<DS.BroadcastSetUserCashEvent>(OnBroadcast);
    EventHub.Instance?.UnRegisterEvent<DS.BroadcastSetUserRecipePiecesEvent>(OnBroadcast);
    EventHub.Instance?.UnRegisterEvent<DS.BroadcastSetUserPerfectRecipeEvent>(OnBroadcast);
```

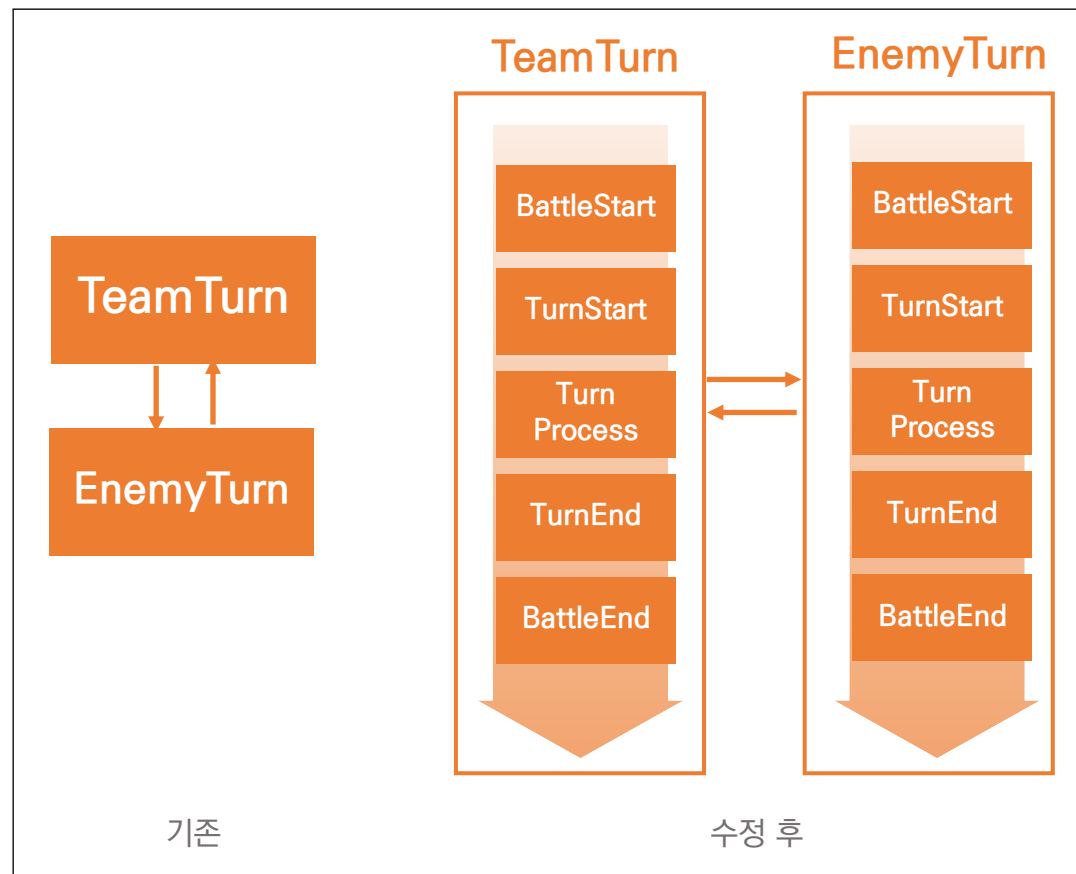
상점 시스템 코드 중 일부

각 UI에서 상황에 따라 이벤트를 통해 View를 표현하는 방식 사용

> 프로젝트 회고

● 프로젝트 진행 중 주요 발생 문제와 대응

문제 이름	턴 진행 구조의 불명확으로 인한 전투 로직 혼선 문제
문제 원인	초기 턴 구조가 TeamTurn / EnemyTurn의 단순 이분 구조에 머물러 있음 스킬 발동, 사망 처리, 스폰, 타일 효과 등이 같은 구간에 뒤섞여 처리 그 결과 스킬 트리거 기준이 불분명
문제 해결 대상	기획팀
해결 방법	전투 흐름을 BattleStart → TurnStart → TurnProcess → TurnEnd → BattleEnd 5단계 Stage 구조로 명확히 분리 TurnStart / TurnProcess / TurnEnd에 Team / Enemy 구분값 추가 아군/적군 기준 스킬 트리거 독립 실행



턴 진행 구조

> 프로젝트 회고

● 프로젝트 진행 중 주요 발생 문제와 대응

문제 이름	아웃게임 초기 MVP 스코프 과대 설정 문제
문제 원인	초기 설계 단계에서 개발 인력·숙련도·제작 속도를 정확히 반영하지 못한 채 기능 수를 기준으로 스코프를 설정하여 실제 구현 가능 범위를 초과한 기획이 누적됨
문제 해결 대상	기획팀
해결 방법	개발팀과의 논의 후 공수 재산정 실제 구현 가능한 개발 볼륨을 다시 정의 그 기준에 맞춰 아웃게임 기획서 전면 리빌딩 모든 기능을 MVP 필수 기능, 후순위 기능, 퀄리티업 디테일로 명확히 재분류 기획서 서식 자체를 우선순위 기반 구조로 재설계하여 개발 순서가 문서만 봐도 명확히 드러나도록 개선

10.2 레벨 업(후순위)
10.3 획득 경로
11. 모험보내기 시스템 (후순위)
11.1 핵심
11.2 맵/기본 보상
11.3 보상 가중치 계산식
11.4 UI 흐름(예시)
12. 상점 시스템
12.1 상점 탭 구성
12.2 상점 콘텐츠 구성
12.3 콘텐츠 상세 정의
12.4 특수 아이템 추가설명
12.5 확률형 아이템 보상 가이드 (후순위)
12.6 구매 이력 확인 (후순위)
12.7 환불 정책 (후순위)

아웃게임 기획서 목차 일부

변경 날짜	문서 버전	변경 사항
10/23	v0.1	- 도넛 반죽 레시피 시스템 식재 - [프로그래밍 시 도넛 제작에 필요한 재료 수의 인스펙트(노출) 명시] - 레시피 제공 구조 및 조건 명확하게 정의 - 플레이 확장을 최대 30칸까지 가능하도록 변경 - 도넛 미지 강화 시, 필요로 하는 특정 레벨의 재료 도넛이 제작되어 있지 않더라도 하위 레벨의 도넛들이 필요로 하는 특정 레벨의 재료도넛을 만들 수 있을 만큼 존 재한다면, 그 도넛들을 모두 소모시키고 강화하는 시스템 추가 (재료 도넛 자동 생산 시스템 이라고 명명) - 재료 도넛 발급 시스템 정의 - [데이터] - 도넛 미지] 볼륨 값 변경 - 마녀 제방사의 특수도넛 및 재료업 시스템 정의 - 도넛 레벨에 따른 스코프변화 식 정의
10/24	v0.2	- 업적 시스템 구조 변경 (업적의 성질에 따라 트리 구조, 획득재료, 레시피조건, 마녀 제방사 등을 획득 가능) - 도넛 생산 시 생산소요시간(5분)을 고정 → 변동 가능성 있도록 정의 - 상점에 다이아(유물제작) 개발 도입 - 마녀 제방사는 공돈(공물제)으로 구매 불가하도록 재정의 - MMR 시스템 구조 변경(전성중) - 도넛 강화 시 공돈 소모 개념 추가
10/27	v0.3	- 상점 판매 품목 구체화
10/28	v0.4	- 도넛 제작 - 트레이 구조 변경 - 상점 규칙 디테일 보완 - PvP 스테이지 완료 보상 테이블 작성
10/29	v0.5	- UI 부가 설명 목차 작성 - pvp 액션 설정 및 설명 추가 - 인벤토리 설명 추가
10/31	v0.6	- 도넛 발급 트리 구조 변경 - 상점 아이템 제거 값 정의 - 인벤토리 - 도넛 창힐 버튼 정의 - 인벤토리의 인 제방 정의 - 로그인 시스템 정의(전성중) - 도넛 제작 시 스코프 min-max 값 사라지고 지정값으로 고정함 - 인벤토리 구조 변경 (영역 필터 추가, 같은 도넛 겹치짐, 레벨과 개수 표시) - 도넛 미지 강화 UI 상세설명 추가

아웃게임 기획서
문서변경사항 일부

기획서 목차에 후순위 기능은 명확하게 표시,
퀄리티업 디테일 등 주요 스코프 이외의 기능은 단순한 로직으로 리빌딩 후 변경사항 기록